🎓 Capstone Project

Multi-Agent AI System for IT Support

Project Overview
Students will design, build, and present a **multi-agent Agentic AI system** that provides IT support. The goal is to apply product ownership, UX design, and technical knowledge to create a system that demonstrates real-world readiness.

🛠 Key Steps

1. **Define the Use Case**
   - Work as a “product owner” to specify IT support scenarios (e.g., password resets, software troubleshooting, ticket triage).
   - Document the problem space and identify clear success metrics.

2. **Identify the Agents**
   - Determine roles:
     - **Intake Agent** – understands and classifies the user request.
     - **Knowledge Agent** – retrieves information (RAG + embeddings + vector DB).
     - **Workflow Agent** – executes automations (resetting accounts, checking logs).
     - **Escalation Agent** – routes unresolved cases to humans.

3. **UX Design**
   - Create wireframes/mockups for how users interact with the system.
   - Focus on clarity and ease-of-use (*ChatGPT study and learn sample prompt: clarity and ease-of-use UX design examples*).

4. **System Development**
   - Build using frameworks (Ideally LangGraph but allow no-code options with customization).
   - Leverage project “help days” for debugging and coaching.

5. **Testing & Validation**
   - Validate agent behavior with real IT support scenarios.
   - Track accuracy, response time, and user satisfaction.

6. **Presentation**
   - Deliver a demo of the system.
   - Share design decisions, technical implementation, and lessons learned.
   - Reference the sample Demo slide deck, it is for reference ONLY

🔍 Special Considerations

- **RAG Integration**
  - Show how embeddings + vector stores (e.g., Pinecone, Weaviate) ground the agent with IT knowledge.
  - Highlight best practices: privacy, reducing hallucinations, ensuring accurate context retrieval.

- **Workflow Automation**
  - Identify repeatable IT processes to automate (ticket creation, system checks, log analysis).
  - Explore orchestration (via tools, code, or graph-based flows).

- **Integration via MCP**
  - Use VS Code as a client connected to GitHub or Jira via **Model Context Protocol**.
  - Demonstrate how MCP standardizes tool access compared to traditional APIs.

- **Multi-Agent Collaboration**
  - Show how domain-specific agents work together (knowledge vs. workflow vs. escalation).
  - Emphasize scalability and separation of concerns in enterprise environments.

- **Industry Awareness**
  - Research current vendors (like **Glean** for enterprise search/AI support).
  - Position your project as a “lightweight alternative” or a prototype of enterprise-grade solutions.

🎯 Learning Outcomes

By the end of the project, students will be able to:

- Apply **product ownership** thinking to an AI system.
- Demonstrate **multi-agent orchestration** in a real-world IT context.
- Implement **RAG, workflow automation, and MCP integration**.
- Communicate design and technical trade-offs in a **professional presentation**.